

TANGENT STREAMLINED ARCHITECTURE LIFECYCLE

Ingestion-to-Reader System Engineering Specification

1. EXECUTIVE ARCHITECTURAL SUMMARY

Our team's primary engineering objective is the refactoring of **Tangent**, moving from an 'accidental complexity' prototype of stacked heuristics to a streamlined, highly performant systems architecture. The prototype demonstrated viable product metrics but suffered from extreme database scaling walls, parallel evaluation overhead, and procedural deadlock risks. By consolidating the recommendation logic into three clean mathematical pillars, we eliminate arbitrary scoring normalizations, remove serverless infrastructure crons, and establish a bulletproof, easily testable recommendation lifecycle.

To guarantee structural health, the unified system redesign establishes three core mathematical guidelines: (1) Every personal preference and collective system memory scales to a strict $[0.0, 1.0]$ probability scale; (2) A continuous **Engagement Index (E)** replaces parallel geometry-based labels and stepwise attention multipliers; and (3) Dynamic score discounts replace fragile bucketing and shuffler loops. This specification lays out the exact blueprints to build out our refactoring sprint.

Pillar Concept: Simplifying Operations & Restoring Integrity

- Standardized Memory: 0.5 is strictly neutral; linear scoring is preserved without piecewise Rescaling.
- Single-Pass Selection: Removes procedural deadlocks; layout diversity is achieved through mathematical penalty decay.
- Platform Parity: Swift preloader parity on iOS to completely offload JS runtime threads.

2. INGESTION & DATABASE ARCHITECTURE

Tangent runs a scheduled ingestion cycle (**rssCollector**) every 3 hours to fetch articles from 42 verified RSS feeds. At ingestion, new articles are assigned a stable **random_score** float [0, 1), which is refreshed daily to optimize uniform lottery sampling.

The 1 MB Firestore Scaling Wall: In the legacy prototype, a scheduled 6-hour cron compiled up to 2,000 pre-selected candidates into a static, flat array inside a single Firestore document (`system/candidatePool_current`). Firestore enforces a hard 1 MB limit on individual documents. With our current metadata schema density, **the candidate pool document would exceed 1 MB at approximately 1,250 active articles**, throwing write exceptions and breaking the feed generator for all users.

The Streamlined Solution: We completely deprecate the flat candidate-pool document and migrate to **Firestore Subcollections** under the path `system/candidatePools/current/articles/`. By writing candidate documents individually into a subcollection of document references, we can scale to millions of active articles without ever risking a document size crash. Real-time feed requests query the subcollection dynamically on-the-fly using cheap, index-supported queries, eliminating the cron-job serverless infrastructure entirely.

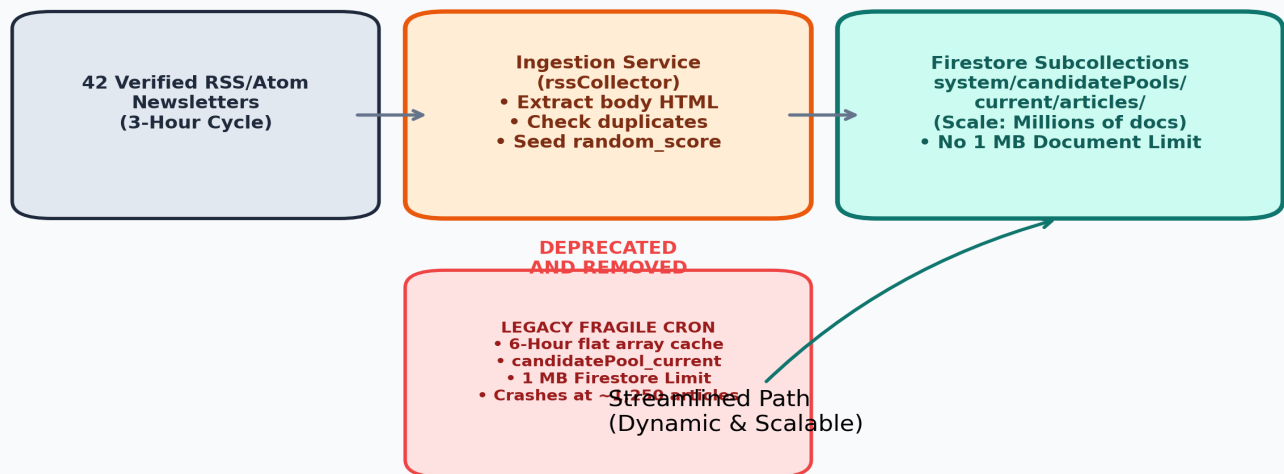


Figure 1: Ingestion Pipeline and Firestore Subcollection Migration

3. PILLAR A: STANDARDIZED MEMORY SCALES & NORMALIZATION

To ensure scoring components are combined predictably, the recommendation engine maps all component scores onto a strict [0.0, 1.0] scale. The legacy prototype used multiple disjointed rulers packed with arbitrary piecewise normalizations. Pillar A standardizes all memories onto a standard 0-to-1 scale:

User Preferences (P): Instead of the legacy [0.1, 5.0] scale—where a neutral, unobserved category of 1.0 was compressed down to a weak 18% score (0.9/4.9)—we initialize all preference weights to **0.5 as strictly neutral**. Positive events push the weight toward 1.0; negative events drift it toward 0.0. Scoring personal fit becomes a direct, zero-normalization calculation: $P = w_{cat} * CatWeight + w_{pub} * PubWeight$. This removes 'two sources of truth' bugs where dashboard clamps differed from hardcoded scorer limits.

Trending Heat (T): Instead of dividing raw trending scores (S) by a hardcoded cap (like 50), which is vulnerable to traffic spikes, we run scores through a standard self-calibrating saturation sigmoid: $T = S / (S + k)$, where k is the half-saturation constant (e.g., $k = 10$). T naturally scales between 0 and 1, regardless of total app traffic.

Publisher Quality (Q): We store publisher quality directly as a [0.0, 1.0] trust index (seeded at 0.8), removing any need for piecewise scaling (e.g. 0.8 -> 0.75, 1.0 -> 1.0). At write-time, the syncBehaviorEvents Cloud Function enforces strict clamping on updates, preventing values from escaping boundaries.

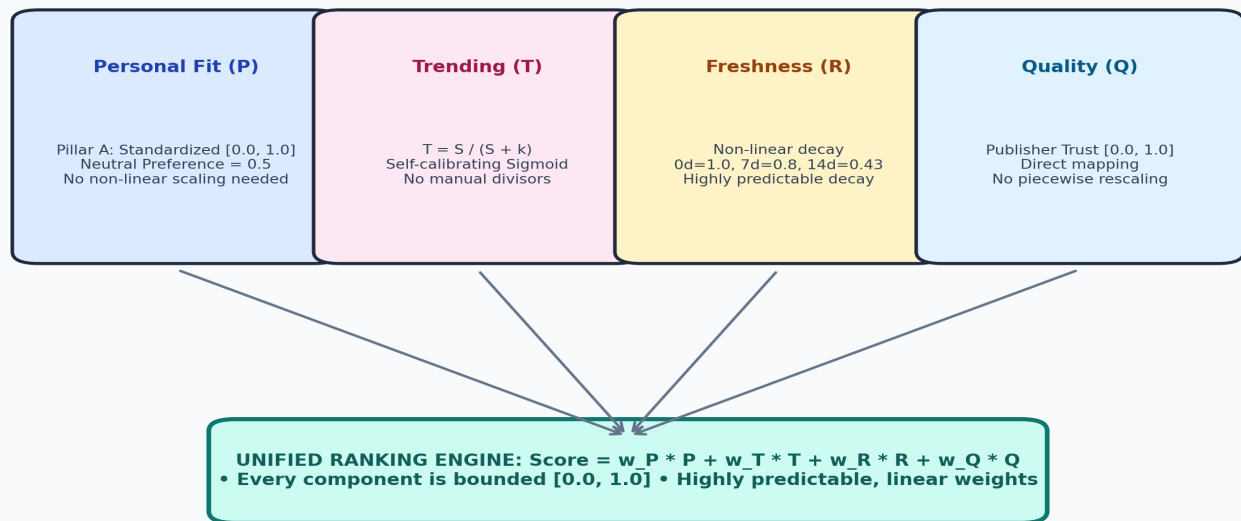


Figure 2: Component Normalization on a Standard [0.0, 1.0] Ruler

4. PILLAR C: SINGLE-PASS DISCOUNTED PENALTY SELECTION

The legacy shuffler was highly convoluted: scoring candidates, bucketing them into High/Mid/Tail tranches, randomly selecting from those buckets, and then running a heavy post-processing 'replace-and-shuffle' loop to enforce category limits and spacing. This procedural approach frequently entered deadlock loops when confronted with highly concentrated preferences or small candidate pools, creating severe latency spikes.

Pillar C replaces the entire tranche-and-replace loop with a standard **Discounted Penalty Selection** algorithm (similar to Maximal Marginal Relevance), executed in a single sequential greedy pass:

1. Score Candidates: We score all 200 candidates using a unified, predictable formula: $Score = w_P * P + w_T * T + w_R * R + w_Q * Q$. The highest-scoring article is locked at position 0 as the Dashboard Hero.

2. Card-by-Card Selection: We compile the remaining 29 slots one-by-one. Each time an article is selected, we dynamically apply temporary penalty multipliers to all remaining candidates that share its publisher or category. If a Politics card is selected, we temporarily multiply the scores of all remaining Politics cards by a category penalty (e.g., 0.6). If a Noahpinion card is selected, we multiply remaining Noahpinion cards by a publisher penalty (e.g., 0.4).

This elegant design naturally interleaves categories and enforces spacing without complex swapping loops. If a user loves Politics intensely, they will still receive multiple Politics cards because their high base personalization overcomes the penalty. Variety strength is easily controlled by a single slider on the dashboard.

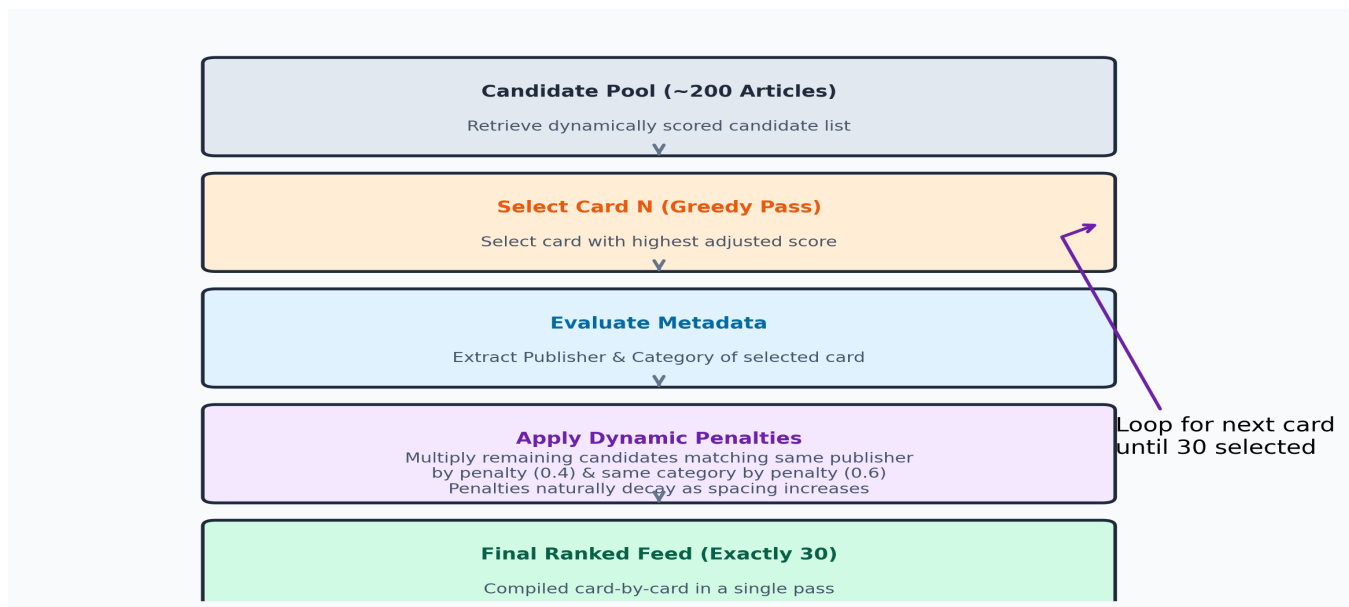


Figure 3: Single-Pass Greedy Selection and Dynamic Variety Penalties

5. PILLAR B: UNIFIED TELEMETRY & ENGAGEMENT

Our prototype evaluated reader behavior through two separate modules: (1) a geometry-only classifier yielding 'thorough', 'shallow', or 'quick exit' labels, and (2) an Attention-Factor (A) model that scaled database updates based on words-per-minute (WPM). This dual-gated pipeline was structurally fragile. For instance, a 'thorough read' (scroll depth $\geq 70\%$) could have its weight update completely zeroed out if the speed crossed an arbitrary 1,750 WPM threshold (a 'fling'), leaving silent database update dead zones.

Pillar B collapses both steps into a single, continuous **Engagement Index (E)** calculated on article exit: $E = \text{Scroll Depth} * \text{Pace Penalty}$.

The **Pace Penalty** is modeled as a smooth, continuous sigmoid curve based on reading speed (WPM):

- WPM ≤ 300 : Penalty = 1.0 (Full reading speed)
- WPM = 600: Penalty = 0.5
- WPM ≥ 1200 : Penalty = 0.0 (Fling threshold)

Every database update (category preferences, publisher quality, and trending heat) is simply multiplied by the single E value. If a user scrolls 80% of an article but flings through it, E is calculated as 0.15. The database naturally applies a tiny 15% weight increment, entirely removing rigid 'thorough vs. shallow vs. fling' labels. WPM calibration is updated using an 80/20 rolling average, strictly guarded by a human-plausibility band of [80, 600] WPM and a 150-word floor to protect the baseline from skims.

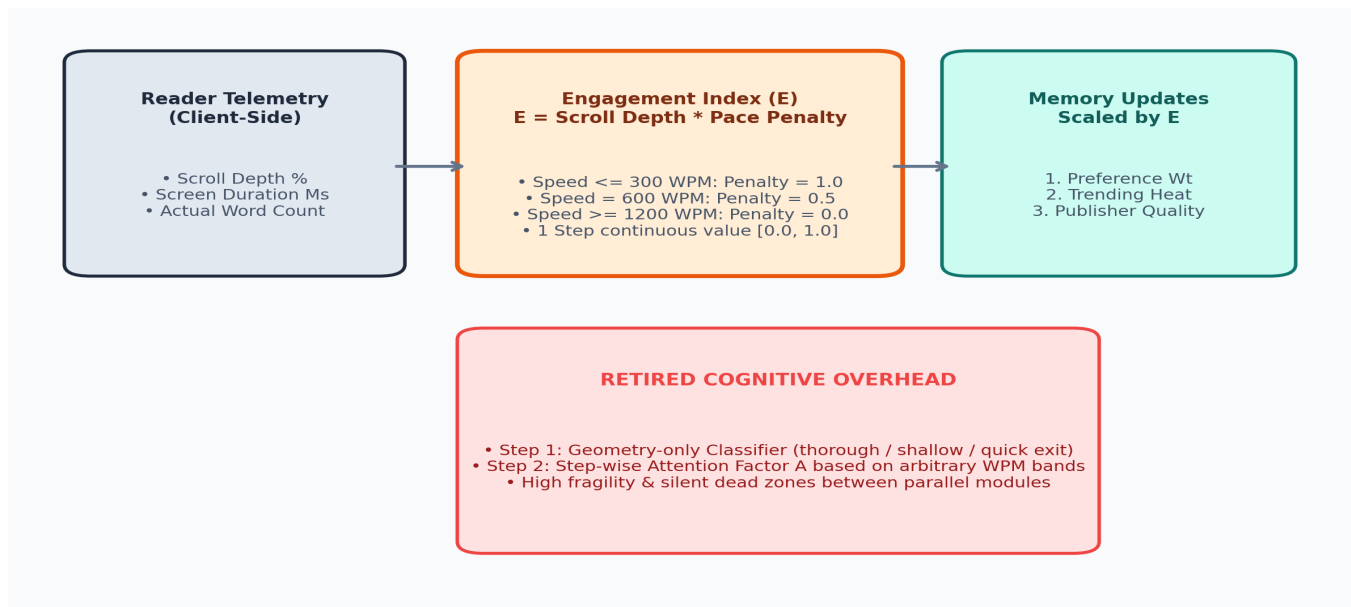


Figure 4: Unified Engagement Index & Atomic Memory Updates

6. CLIENT-SIDE PARITY & DATABASE SCHEMAS

To achieve a seamless swipe experience, Tangent utilizes background preloading. On Android, a custom Kotlin local Expo module (modules/tangent-rss-parser) streams XML feeds on a background serial worker, maintaining a rolling five-upcoming-article buffer in memory. This prevents the single React Native JavaScript runtime thread from blocking, protecting UI frame rates.

The iOS Performance Gap: Currently, iOS devices lack this local module and fall back to the JavaScript-based *fast-xml-parser*, suffering from visible scroll lag and loading spinners during rapid swiping. We must implement a Swift-based local Expo module (modules/tangent-rss-parser/ios/) that implements the exact same native background streaming APIs. This guarantees perfect performance parity across both platforms before store deployment.

REFACTORED ARCHITECTURAL MEMORIES SUMMARY

Memory	Temporal Layer	Standardized Scale	Update Delta & Decay Mechanics
User Preferences (P)	Personal, Slow-Evolving	Init at 0.5, [0.0, 1.0]	Category $\pm \Delta * 0.08$, Publisher $\pm \Delta * 0.16$. Nightly decay: drifts 0.5% back to 0.5.
Trending Heat (T)	Collective, Fast-Decaying	Sigmoid $T = S / (S + k)$	Save (+3.0), Like (+2.0), Read ($E * +1.5$). Nightly decay: cools by 9.057% (halves weekly).
Publisher Quality (Q)	Collective, Slow-Evolving	Direct clamp [0.2, 1.0]	Save (+0.010), Like (+0.005), Read ($E * +0.005$), Swipe (-0.010), Quick-Exit (-0.005).

DATABASE FIRESTORE SYSTEM SCHEMA

Field Name	Firestore Type	Role & Clamping Bounds
userId	string	Matches Document ID (Auth UID)
categoryWeights	Record	Learned preferences. Initialized at 0.5, bounded [0.0, 1.0]
publisherWeights	Record	Learned publisher weights. Initialized at 0.5, bounded [0.0, 1.0]
averageWpm	number	Rolling 80/20 average. Guarded by [80, 600] WPM calibration filter
totalReadTimeMs	number	Cumulative foreground active duration, excluding background pauses

Sprint Implementation Alignment

By aligning our database layout and client parsers to this unified blueprint, our team permanently resolves the Firestore 1 MB document limitation, simplifies the scoring math, eliminates server-side crons, and ensures iOS users experience the same buttery-smooth 60fps swipe experience as Android.